

个人信息

- 专业：计算机科学与技术
- 岗位：Go开发工程师
- GitHub：

专业技能

- 熟悉数据结构和算法，具备良好的算法思想和思维逻辑和复杂场景下的算法设计经验，熟悉高级数据结构的使用
- 熟悉Go语言，了解map、slice、channel、GMP模型、gc垃圾回收、内存逃逸等底层原理
- 熟悉 Gin Web 框架，了解 Argo Workflows 并用于定时任务调度与编排，具备基于 Eino 构建 Agent 工作流的实践经验
- 熟悉 MySQL、MongoDB、Redis，具备 SQL 调优与索引设计经验
- 熟悉消息队列 Kafka 及其在异步解耦、削峰场景下的使用
- 熟悉 Linux、Docker、K8S，了解 CI/CD 流程
- 熟练运用 Claude Code、Cursor 等 AI 原生开发工具，擅长编写高质量 Prompt、维护项目级 CLAUDE.md 及优化上下文配置
- 熟悉跨境支付业务链路，理解清算、结算、收单、对账、请款等核心流程，对支付系统的资金一致性与合规要求有实际落地经验

工作经历

公司：境外支付	职位：后端	时间：2024.1-2026.4
公司：电商平台	职位：后端（实习）	时间：2023.8-2024.1
公司：网络安全	职位：后端（实习）	时间：2023.4-2023.7

教育经历

2020.09 - 2024.06	浙江**大学	计算机科学与技术	本科
在校经历：CET-6	学年奖学金	RoboCom 机器人开发编程	ACM

项目经历

项目：境外支付Sass

项目角色：后端

项目介绍：跨境支付服务商，服务覆盖 10+ 国家与地区，日交易量级百万级，涉及收单、清算、结算、对账、请款等核心模块。主要负责清结算与对账相关模块的开发与维护。

- 参与清结算模块 oncall，负责线上问题的定位与止损，保障每日百万级交易的资金准确性与系统稳定性
- 针对卡组织 BIN 导致的任务耗时过长问题，结合协程分片与贪心合并策略优化处理效率，任务执行时间从 10h 降至 30min
- 基于 errgroup 重构下游服务调用链，采用 DAG 拓扑排序处理存在依赖关系的多下游调用，实现无依赖节点并行执行，核心接口 RT 从 1.8s 降至 400ms
- 改造 reporter 服务为常驻进程并设计与现有调度兼容的任务分发机制，支持多调度方式共存，扩展文件任务灵活性
- 针对跨境支付合规要求，落地文件数据的全流程加密存储，维护内部平台解密工具，降低人工处理成本
- 针对日报报表任务全量扫描交易明细导致执行过长的问题，引入中间聚合层做分批预统计，报表生成时间缩短 70%

- 维护项目级 CLAUDE.md，将请款文件生成的金额精度、字段规范、资金流向等业务规则沉淀为 AI 约束，显著降低 AI 生成代码的人工 review 成本与合规风险
- 基于 Eino 框架构建对账测试工具，自动生成边界场景并驱动交易全链路执行（交易发起 → 清算 → 对账结果校验），补齐单边账、金额差错、跨日延迟场景，核心对账模块单元测试覆盖率提升至 99%

项目：银行入网

项目角色：后端（核心开发）

项目介绍：面向恒生合作机构的商户入网平台，承接从资料提交、合规审核、协议生成、费率配置到下游支付渠道开户的全流程入网能力，是支付交易链路的前置环节。

- 设计并落地通用工作流引擎：抽象节点的 DAG 流转，支持资料提交、初审、复审、渠道开户、激活等环节按配置编排，新增流程节点无需改码
- 设计 SLA 超时预警机制：按 FunctionMenu、TaskType等多级评分匹配 SLA 配置，基于任务创建时间实时计算 ExpectedDate 与 Priority，支撑入网时效监控
- 基于 HistoryEntry 完整记录每次节点进入 / 审批 / 回退 / 表单变更的操作人、时间戳、前后值，支撑入网全链路审计追溯与合规要求
- 构建可配置资料校验规则，将原本散落在代码中的硬编码校验抽象为 JSON 可配置规则，支持不同渠道 / 机构差异化策略，新增校验规则无需改码

项目：电商平台

项目角色：后端（实习）

项目介绍：电商平台主站交易链路 with 基础脚手架维护，负责订单、退款、限流等核心模块的需求开发与稳定性优化，日单量十万级。

- 周期购需求，复用原有的正逆向逻辑，新增子订单依附及零元购通用逻辑，简化业务重复操作
- 优化缓存限流策略，使用漏斗算法及Aop思想实现接口级限流
- 使用分布式锁实现底层接口的幂等性，减少上游业务的复杂逻辑的考虑。
- 优化应用脚手架的代码规范，包括异常处理，自研框架应用等，降低新建应用成本80%
- 使用异步编排的方式重构获取数据接口，实现页面的预加载，提高用户体验
- 对接第三方应用，复用主站正向逆向逻辑，实现新增三方的可插拔式开发